Лекц 3. Сийрэг матрицын технологи: хадгалах схем, оновчтой эрэмбэлэлт, хүчин зүйлийн хүснэгт

Багш: Тэнхимийн эрхлэгч, доктор (Ph.D), дэд профессор Ш.Гантөмөр · Хугацаа: 2 цаг

Лекцийн зорилго. Цахилгаан сүлжээний матрицын сийрэг шинжийг тоон үзүүлэлтээр тодорхойлох; сийрэг матрицыг координатын (COO), шахсан мөрийн (CSR) ба холбоост жагсаалтын схемээр хадгалах алгоритмыг байгуулах; задаргааны үеийн дүүргэлтийг графаар тайлбарлаж Тиннигийн оновчтой эрэмбэлэлтийн (1, 2, 3-р схем) алгоритмыг судлах; сийрэг LU ба хүчин зүйлийн хүснэгт, сийрэг векторын аргыг программчлах; бүх алгоритмын дэлгэрэнгүй блок схемийг гаргах.

Түлхүүр үг: сийрэг матриц, сийрэгшил, COO, CSR, холбоост жагсаалт, дүүргэлт, элиминацийн граф, оновчтой эрэмбэлэлт, Тиннигийн схем, хамгийн бага зэрэг, хүчин зүйлийн хүснэгт, сийрэг вектор, бэлгэдлийн задаргаа.

1. Сүлжээний матрицын сийрэг шинж

Цахилгаан сүлжээний зангилаа нь дунджаар 2–4 зангилаатай шууд холбогддог. Иймд n зангилаатай Y матрицын мөр бүрт ердөө 3–5 тэгээс ялгаатай элемент (диагональ ба хөршүүд) байна. **Сийрэгшил** ρ = (тэг элементийн тоо)/n² нь 1000 зангилаат сүлжээнд ≈ 99.6% болно. Тэг элементийг хадгалж, тэдгээрээр үйлдэл хийх нь санах ой ба хугацааны асар их үрэлгэн байдал юм:

Сүлжээ	n	Салаа m	nnz = n + 2m	Сийрэгшил, %	Дүүрэн санах ой	Сийрэг санах ой
IEEE 14	14	20	54	72.4	1.6 КБ	0.9 КБ
IEEE 118	118	186	490	96.5	111 КБ	8 КБ
Монголын ТЭХС (≈)	450	620	1 690	99.2	1.6 МБ	27 КБ
Европын сүлжээ (≈)	10 000	15 000	40 000	99.96	800 МБ	0.6 МБ

Хүснэгт 3.1. Сүлжээний матрицын сийрэгшил (8 байтын тоо, CSR-д 12 байт/элемент)

Сийрэг матрицын технологийг 1960-аад онд У.Тинни (Bonneville Power Administration) цахилгаан сүлжээний тооцоонд зориулж боловсруулсан бөгөөд Ньютоны аргыг том сүлжээнд хэрэглэх боломжийг анх олгосон. Түүний гурван санаа: (1) зөвхөн тэгээс ялгаатай элементийг хадгалах; (2) задаргааны дүүргэлтийг багасгах эрэмбэлэлт; (3) задаргааг дахин ашиглах хүчин зүйлийн хүснэгт.

2. Хадгалах схем

2.1. Координатын схем (COO)

Тэгээс ялгаатай элемент бүрийг (мөр, багана, утга) гурвалаар хадгална. Сүлжээний өгөгдөл (салааны жагсаалт)-аас шууд гарах тул матриц бүрдүүлэх үед тохиромжтой, гэвч мөрөөр хандахад эрэмбэлэлт шаардана. Санах ой: 3·nnz.

2.2. Шахсан мөрийн схем (CSR)

Гурван массив: val — утгууд мөрөөр дараалан, col — баганын дугаар, rp — мөр бүрийн эхлэлийн заагч (n + 1 урттай). i-р мөрийн элементүүд val[rp_i] … val[rp_{i+1} − 1]. Матриц-вектор үржвэр y = Ax-ийн алгоритм:

$$y_i = \Sigma_{p\,=\,rpi}^{rpi+1-1}\, val_p \cdot x_{col_p}, \qquad i = 1, \ldots, n \tag{3.1}$$

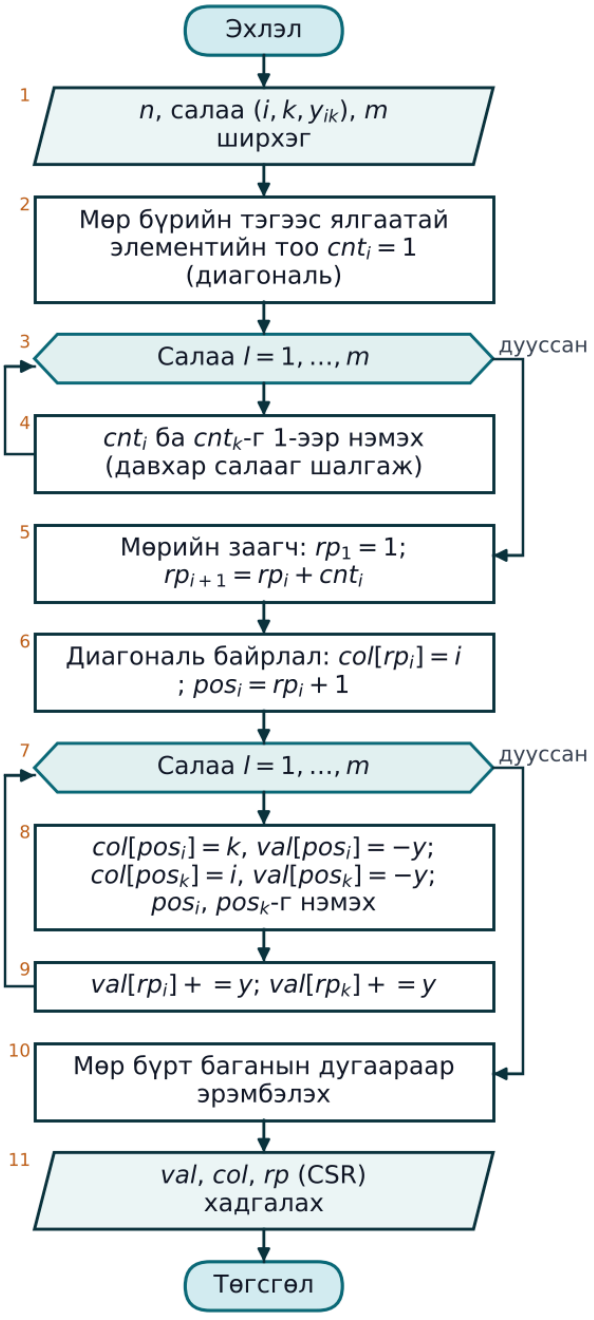
CSR нь санах ой (2·nnz + n + 1) ба хурдны хувьд хамгийн түгээмэл; SciPy (scipy.sparse.csr_matrix), MATLAB (CSC — баганаар), Intel MKL зэрэг бүгд хэрэглэдэг. Сул тал нь шинэ элемент (дүүргэлт) оруулахад массивыг шилжүүлэх шаардлагатай тул задаргаанаас өмнө **бэлгэдлийн задаргаагаар** дүүргэлтийн байрыг урьдчилан тодорхойлно.

2.3. Холбоост жагсаалт

Элемент бүр (утга, багана, дараагийн элементийн заагч) хадгалж, мөр бүрийн эхний элементийн заагчтай. Шинэ элемент оруулах O(1) тул задаргааны үед дүүргэлтийг шууд нэмэхэд тохиромжтой; Тиннигийн анхны программууд энэ схемийг хэрэглэсэн. Сул тал — санах ойн тасархай хандалт удаан.

> **Жишээ 3.1.** Таван зангилаат сүлжээ (DC загвар, урвал о.н): x_{12} = 0.1, x_{13} = 0.2, x_{14} = 0.25, x_{15} = 0.2, x_{23} = 0.125. B матрицыг бүрдүүлж CSR хэлбэрээр хадгал.
>
> *Бодолт.* B = [24 −10 −5 −4 −5; −10 18 −8 0 0; −5 −8 13 0 0; −4 0 0 4 0; −5 0 0 0 5].
>
> nnz = 5 + 2·5 = 15 (25-аас), сийрэгшил 40%.
>
> val = [24 −10 −5 −4 −5 | −10 18 −8 | −5 −8 13 | −4 4 | −5 5].
>
> col = [1 2 3 4 5 | 1 2 3 | 1 2 3 | 1 4 | 1 5].
>
> rp = [1 6 9 12 14 16] — 3-р мөр нь val[9..11].
>
> Шалгалт: y = Bx, x = (1; 1; 1; 1; 1) ⇒ мөрийн нийлбэр бүр 0 (газартай холбоогүй сүлжээ) ✓.

Зураг 3.1-д салааны жагсаалтаас CSR бүтцийг хоёр дамжлагаар (эхлээд мөр бүрийн элементийн тоо, дараа нь утга) байгуулах алгоритмыг үзүүлэв. Энэ арга O(n + m) хугацаатай.



Зураг 3.1. Салааны жагсаалтаас CSR бүтэц байгуулах блок схем

3. Задаргааны дүүргэлт ба граф

Гауссын k-р алхамд $a_{ij} - m_{ik}a_{kj}$ үйлдэл нь $a_{ij} = 0$ боловч $a_{ik} \neq 0$ ба $a_{kj} \neq 0$ үед шинэ тэгээс ялгаатай элемент — **дүүргэлт** (fill-in) үүсгэнэ. Графын хэлээр: k зангилааг хасахад түүний бүх хөршүүдийг хос хосоор нь холбоно (элиминацийн граф). Иймд дүүргэлт нь элиминацийн **дарааллаас** хүчтэй хамаарна.

Жишээ 3.2. Жишээ 3.1-ийн сүлжээнд (1 — «од»-ын төв, 4 хөрштэй) (а) 1, 2, 3, 4, 5 ба (б) 4, 5, 1, 2, 3 дарааллаар задлахад үүсэх дүүргэлтийг ол.

Бодолт. (а) 1-р зангилааг эхэлж хасахад хөршүүд {2, 3, 4, 5} хос хосоороо холбогдоно: (2, 4), (2, 5), (3, 4), (3, 5), (4, 5) — 5 дүүргэлт ((2, 3) аль хэдийн байгаа). Матриц бараг дүүрэн болно.

(б) 4 (зэрэг 1) ба 5 (зэрэг 1)-г эхэлж хасахад нэг л хөрштэй тул дүүргэлтгүй; дараа нь 1 (хөрш 2, 3 — аль хэдийн холбоотой), 2, 3 — **0 дүүргэлт**.

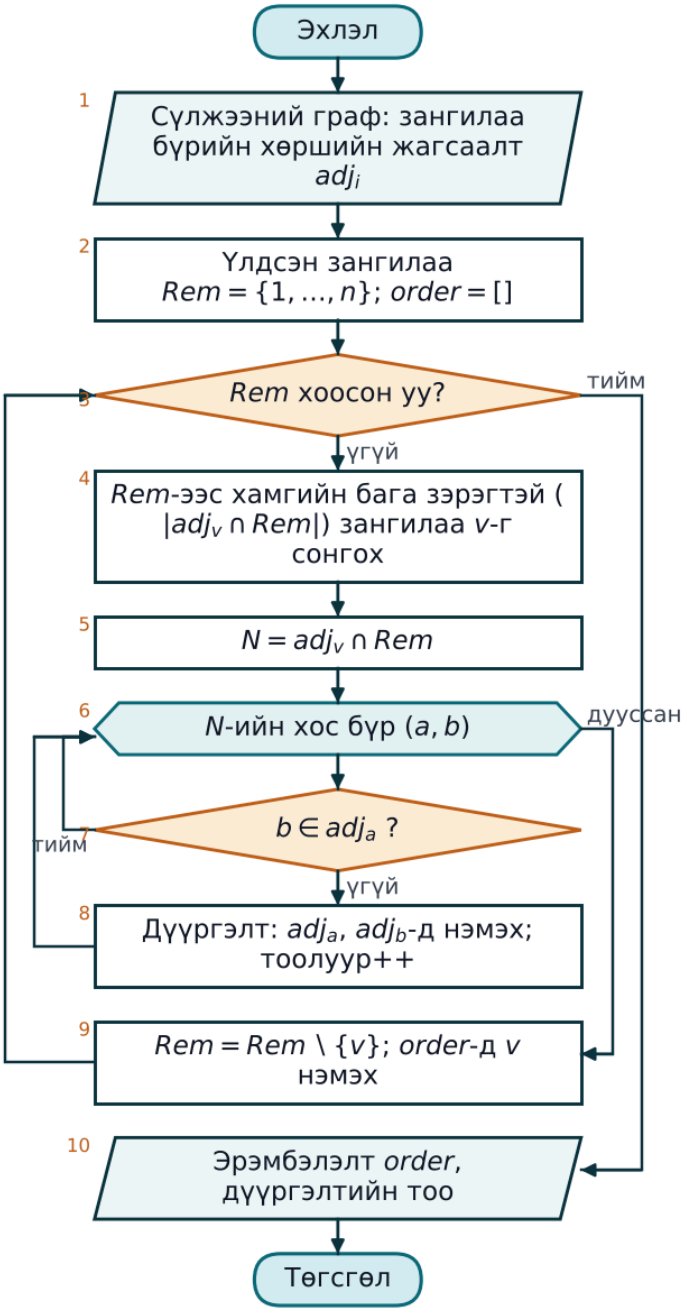
Үйлдлийн тоо: (а) ≈ 30, (б) ≈ 12 үржих үйлдэл; санах ой (а) 25, (б) 15 элемент.

Дүгнэлт: «бага холбоотой зангилааг эхэлж хас» гэсэн энгийн дүрэм дүүргэлтийг эрс бууруулна.

4. Тиннигийн оновчтой эрэмбэлэлт

Хамгийн бага дүүргэлттэй дарааллыг олох нь NP-бүрэн бодлого тул эвристик схем хэрэглэдэг. Тиннигийн гурван схем:

1. **1-р схем (статик зэрэг).** Задаргааны өмнө зангилааг анхны зэргээр (хөршийн тоо) өсөхөөр эрэмбэлнэ. Хамгийн хурдан, үр дүн дунд.
2. **2-р схем (хамгийн бага зэрэг, minimum degree).** Алхам бүрт **одоогийн** (өмнөх дүүргэлтийг тооцсон) хамгийн бага зэрэгтэй зангилааг сонгоно (Зураг 3.2). Практикт хамгийн өргөн хэрэглэгддэг; AMD (approximate minimum degree) нь түүний хурдан хувилбар.
3. **3-р схем (хамгийн бага дүүргэлт).** Алхам бүрт хасахад хамгийн бага дүүргэлт үүсгэх зангилааг сонгоно. Дүүргэлт хамгийн бага боловч сонголт бүрт бүх зангилааг шалгах тул удаан.



Зураг 3.2. Тиннигийн 2-р схем (хамгийн бага зэрэг)-ийн блок схем

Жишээ 3.3. Зургаан зангилаат сүлжээ: салаа (1, 2), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (2, 6), (3, 5), (3, 6), (4, 5), (5, 6). Анхны дараалал, 2-р схем ба «муу» дараалал (2, 5, 1, 3, 4, 6)-ын дүүргэлтийг харьцуул.

Бодолт. Анхны зэрэг: d = (3, 5, 3, 3, 5, 3).

Анхны дараалал 1…6: 1-ийг хасахад хөрш {2, 4, 5} — бүгд холбоотой, дүүргэлтгүй; 2-ыг хасахад {3, 4, 5, 6}: (3, 4), (4, 6) — 2 дүүргэлт.

2-р схем: 1 (зэрэг 3), дараа нь 4 (одоогийн зэрэг 2), 2 (3), 3 (2), 5, 6 — **0 дүүргэлт**.

«Муу» дараалал (хамгийн их зэргээс эхлэх) 2, 5, …: (1, 3), (1, 6), (3, 4), (4, 6) — 4 дүүргэлт.

1-р схем (статик): 1, 3, 4, 6, 2, 5 — мөн 0 дүүргэлт (энэ жижиг жишээнд).

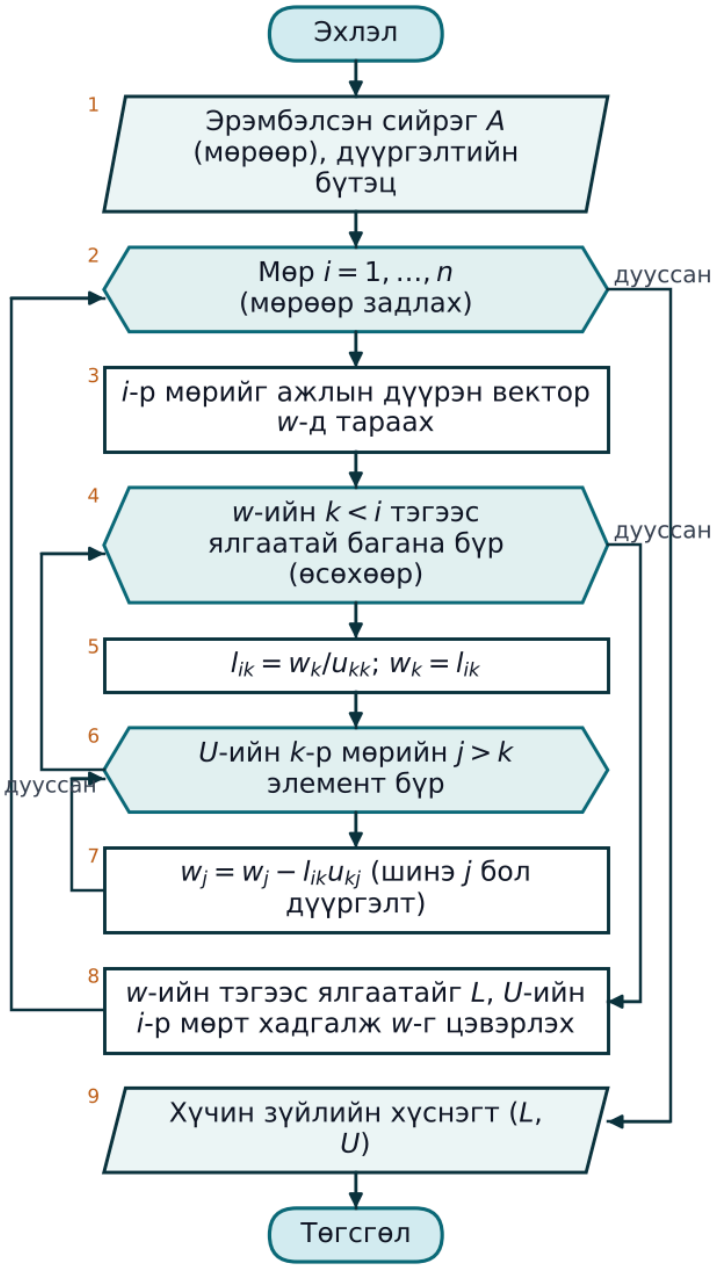
Дараалал	Дүүргэлт	L + U-ийн nnz	Тайлбар
1, 2, 3, 4, 5, 6 (анхны)	2	32	Санамсаргүй
1, 4, 2, 3, 5, 6 (2-р схем)	0	28	Оновчтой
1, 3, 4, 6, 2, 5 (1-р схем)	0	28	Статик зэрэг
2, 5, 1, 3, 4, 6 (их зэргээс)	4	36	Хамгийн муу

Хүснэгт 3.2. Жишээ 3.3-ын эрэмбэлэлтийн харьцуулалт

Том сүлжээнд 2-р схемийн үр дүн гайхалтай: 10 000 зангилаат сүлжээнд L + U-ийн тэгээс ялгаатай элемент ≈ 2–3·nnz(Y), харин санамсаргүй дараалалд 10–100 дахин их. Энэ нь Ньютоны аргын давталтыг дүүрэн матрицынхаас 10^3–10^4 дахин хурдасгадаг.

5. Сийрэг LU ба хүчин зүйлийн хүснэгт

Сийрэг задаргааг гурван үе шаттай хийнэ: (1) **эрэмбэлэлт** (Зураг 3.2); (2) **бэлгэдлийн задаргаа** — утгагүйгээр зөвхөн бүтцийг задалж, дүүргэлтийн байрыг тодорхойлж санах ой нөөцлөх; (3) **тоон задаргаа** — бүтэц тогтмол тул Ньютоны давталт бүрт зөвхөн энэ үе шатыг давтана. Зураг 3.3-т мөрөөр (Дулиттл маягаар) тоон задаргаа хийх алгоритмыг үзүүлэв: i-р мөрийг дүүрэн ажлын вектор w-д тарааж, өмнөх мөрүүдээр хасаад, буцааж сийрэг бүтцэд хадгална.



Зураг 3.3. Мөрөөр хийх сийрэг LU задаргааны блок схем

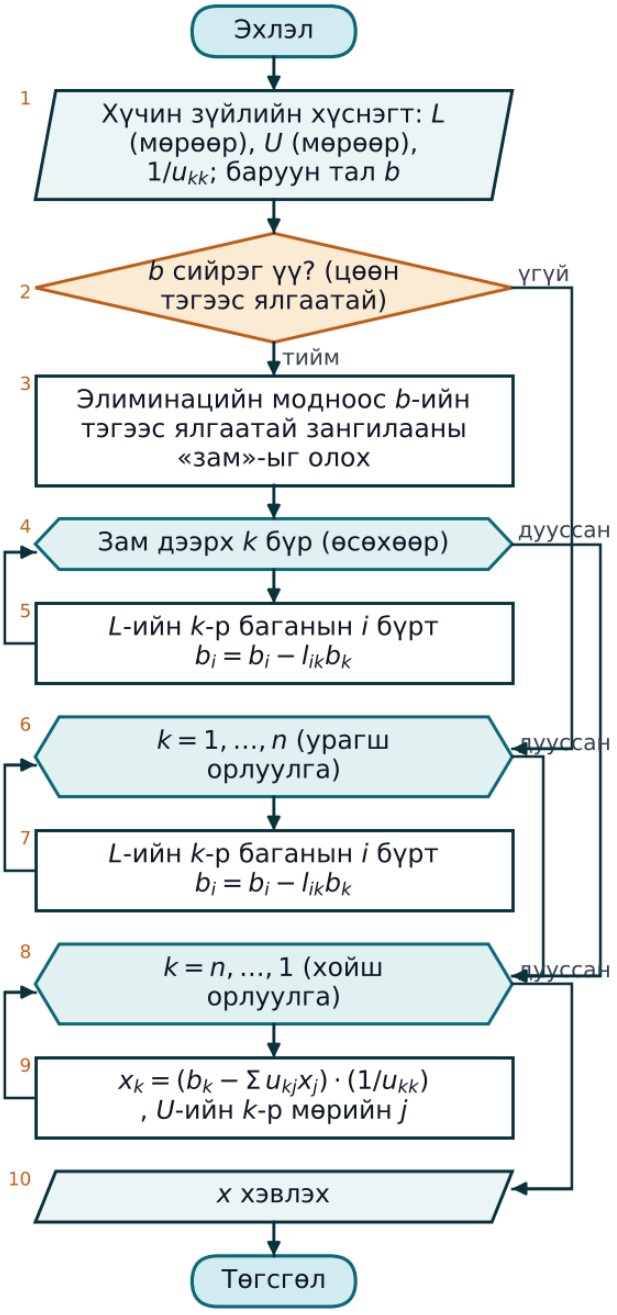
Хүчин зүйлийн хүснэгт (factor table) — L ба U-г (U-ийн диагоналийн урвуу $1/u_{kk}$-г) сийрэг хэлбэрээр хадгалсан бүтэц. Тиннигийн программд баруун тал бүрийг урагш ба хойш орлуулгаар бодоход зөвхөн хүснэгтийн тэгээс ялгаатай элементээр давтана. Сийрэг вектор (баруун тал цөөн тэгээс ялгаатай элементтэй) үед урагш орлуулгыг зөвхөн «замын» (элиминацийн модны) зангилаагаар хийх — **сийрэг векторын арга** нь N−1 шинжилгээ, PTDF тооцоог дахин 10–100 дахин хурдасгана.

```
import numpy as np, scipy.sparse as sp, scipy.sparse.linalg as spl
def build_B(n, lines):
    '''Салааны жагсаалтаас CSR B матриц (COO-оор дамжуулан)'''
    r, c, v = [], [], []
    for i, k, x in lines:
        y = 1/x; i -= 1; k -= 1
        r += [i, k, i, k]; c += [i, k, k, i]; v += [y, y, -y, -y]
    return sp.coo_matrix((v, (r, c)), shape=(n, n)).tocsr()   # давхар элемент
нэмэгдэнэ
def min_degree(n, edges):
    adj = {i: set() for i in range(1, n+1)}
    for a, b in edges: adj[a].add(b); adj[b].add(a)
    rem, order, fill = set(adj), [], 0
    while rem:
        v = min(rem, key=lambda z: (len(adj[z] & rem), z))       # Зураг 3.2, блок 4
        N = sorted(adj[v] & rem)
        for i, a in enumerate(N):
            for b in N[i+1:]:
                if b not in adj[a]: adj[a].add(b); adj[b].add(a); fill += 1
        rem.discard(v); order.append(v)
    return order, fill
B = build_B(5, [(1,2,.1), (1,3,.2), (1,4,.25), (1,5,.2), (2,3,.125)])
print(B.indptr, B.indices, B.data)            # CSR массивууд
lu = spl.splu(B[1:, 1:].tocsc(), permc_spec='MMD_AT_PLUS_A')   # хамгийн бага зэрэг
print(lu.L.nnz + lu.U.nnz, lu.solve(np.array([0.6, -0.9, -0.5, 0.0])))
```

Код 3.1. CSR, хамгийн бага зэргийн эрэмбэлэлт ба SciPy-ийн сийрэг LU

5.1. Хүчин зүйлийн хүснэгтээр бодох ба сийрэг вектор

Зураг 3.4-т хүчин зүйлийн хүснэгтээр баруун талыг бодох алгоритмыг үзүүлэв. Баруун тал b нь цөөн тэгээс ялгаатай элементтэй (жишээ нь нэг зангилаанд тарилт) бол урагш орлуулгад зөвхөн **элиминацийн модны** зам дээрх зангилааг боловсруулна: k зангилааны эцэг нь L-ийн k-р баганын хамгийн бага мөрийн дугаар (k-аас их). Энэ «замын» урт нь ихэвчлэн log n эрэмбэтэй.



Зураг 3.4. Хүчин зүйлийн хүснэгтээр урагш, хойш орлуулга хийх (сийрэг векторын сонголттой) блок схем

Жишээ 3.4. Жишээ 3.1-ийн сүлжээнд 1-р зангилааг баланслагч гэж B′ (2, 3, 4, 5-р зангилаа)-ийг 4, 5, 2, 3 дарааллаар задалж, хүчин зүйлийн хүснэгтийг гарга. 5-р зангилаанд P = −1 (бусад 0) үед θ-г ол.

Бодолт. B′ = [18 −8 0 0; −8 13 0 0; 0 0 4 0; 0 0 0 5] (2, 3, 4, 5). 4 ба 5-р зангилаа баланслагчтай л холбогдсон тул B′-д бусадтай холбоогүй — диагональ блок.

Хүчин зүйлийн хүснэгт: 1/u = (1/4; 1/5; 1/18; 1/9.4444); L-ийн цорын ганц диагональ бус элемент $l_{3,2} = -8/18 = -0.4444$ (2, 3-р зангилааны хооронд); U-д $u_{2,3} = -8$.

Сийрэг баруун тал b = (0; 0; 0; −1) (2, 3, 4, 5 дарааллаар) — элиминацийн модонд 5-р зангилаа «үндэс»-тэй шууд холбоотой, зам = {5}.

$\theta_5 = -1\cdot(1/5) = -0.2$ рад; $\theta_2 = \theta_3 = \theta_4 = 0$ — тарилт зөвхөн 1–5 салаагаар баланслагч руу урсана ✓.

Урагш орлуулгад 1 үйлдэл (дүүрэн аргад 6), хойш орлуулгад 1 — сийрэг векторын арга нь энэ тохиолдолд 6 дахин хэмнэлттэй.

6. Үйлдлийн тоо ба хурдны харьцуулалт

Сийрэг задаргааны үйлдлийн тоо нь L-ийн багана бүрийн тэгээс ялгаатай элементийн тооны квадратын нийлбэр: $N_{оп} \approx \Sigma_k\ c_k^2$, c_k — k-р баганын элементийн тоо. Сүлжээний матрицад оновчтой эрэмбэлэлтийн дараа $c_k \approx 3$–6 тул $N_{оп} \approx 20n$ — шугаман хамааралтай! Дүүрэн аргын $n^3/3$-тай харьцуулбал:

n	Дүүрэн LU, үйлдэл	Сийрэг LU (≈ 20n)	Хурдасгалт
100	$3.3\cdot10^5$	2 000	≈ 170
1 000	$3.3\cdot10^8$	$2\cdot10^4$	≈ $1.7\cdot10^4$
10 000	$3.3\cdot10^{11}$	$2\cdot10^5$	≈ $1.7\cdot10^6$

Хүснэгт 3.3. Дүүрэн ба сийрэг LU-ийн үйлдлийн тоо (сүлжээний матриц)

Бодит хэрэгжүүлэлтэд хаягийн шууд бус хандалтын зардал нэмэгдэх тул хурдасгалт хүснэгтийнхээс 5–10 дахин бага, гэвч мянгат зангилаанд мянга дахин хэвээр үлдэнэ. «Горим 5.0» программ мөн ийм схемээр Y ба Якобианыг хадгалж, эрэмбэлэлтийг сүлжээ өөрчлөгдөх үед л дахин хийдэг.

7. Программчлалын зөвлөмж

- Салааны өгөгдлөөс COO-оор бүрдүүлж, давхар элементийг нэгтгэн CSR болго (Код 3.1).
- Эрэмбэлэлт ба бэлгэдлийн задаргааг сүлжээний топологи өөрчлөгдөх үед л давт; Ньютоны давталт бүрт зөвхөн тоон задаргаа.
- Тэгш хэмт бүтцийг ашиглаж зөвхөн дээд гурвалжинг хадгал ($Y = Y^T$).
- Санах ойн тасралтгүй хандалтын төлөө CSR/CSC-ийг холбоост жагсаалтаас илүүд үз.
- Бэлэн номын сан: SciPy (SuperLU, UMFPACK), KLU (сүлжээний тооцоонд тусгайлан зориулсан), Intel MKL PARDISO.

8. Эрэмбэлэлтийн туршилт: тор хэлбэрийн сүлжээ

m × m тор хэлбэрийн сүлжээний DC матрицыг (зангилаа бүр 2–4 хөрштэй) SuperLU-гаар гурван эрэмбэлэлтээр задалж, задарсан хүчин зүйлүүдийн (L + U) тэг биш элементийн тоог харьцуулав:

Зангилаа n	nnz(A)	Байгалийн дараалал	Хамгийн бага зэрэг (MMD)	COLAMD
100	460	1 918	1 160	1 426
400	1 920	15 638	6 944	9 648
900	4 380	53 158	19 296	29 438

Хүснэгт 3.5. Тор сүлжээний задаргааны дүүргэлт (L + U-ийн тэг биш элементийн тоо)

Байгалийн дараалалд (мөр мөрөөр дугаарласан тор) дүүргэлт ≈ $n^{1.5}$ хуулиар өсч, n = 900 үед анхны матрицаас 12 дахин их болсон. Хамгийн бага зэргийн эрэмбэлэлт (Тиннигийн 2-р схемийн орчин үеийн хувилбар) дүүргэлтийг 2.8 дахин бууруулсан. Тэгш хэмтэй матрицад MMD нь баганын эрэмбэлэлтээс (COLAMD) илүү тохиромжтой. Бодит цахилгаан сүлжээ тороос илүү «радиаль» тул дүүргэлтийн харьцаа ихэвчлэн 1.5–3 байдаг.

8.1. Бүтэн ба сийрэг тооцооны хугацаа

Ижил тор сүлжээнд B′θ = P системийг бүтэн матрицын LAPACK (numpy.linalg.solve) ба сийрэг SuperLU (scipy.sparse.linalg.splu)-гаар бодсон хугацаа ба санах ой (энгийн зөөврийн компьютер):

n	Бүтэн: хугацаа, мс	Сийрэг: хугацаа, мс	Бүтэн: санах ой, МБ	Сийрэг (CSC): санах ой, кБ
400	5.5	1.6	1.3	24.6
1 600	83	3.3	20.5	100.5
3 600	673	6.8	103.7	227.5

Хүснэгт 3.6. Бүтэн ба сийрэг шийдлийн хугацаа, санах ой

n 9 дахин өсөхөд бүтэн аргын хугацаа ≈ 120 дахин (n^3-ийн хуулиар $n^{2.2}$) өсөж, сийрэг аргынх ердөө 4 дахин өсөв; n = 3600 үед сийрэг арга 100 дахин хурдан, санах ой 450 дахин бага. Бодит 5000–50 000 зангилаат системд бүтэн арга огт хэрэглэгдэхгүй.

```
' CSR хэлбэрийн комплекс Y матрицыг вектort үржүүлэх: I = Y·U (Гаусс-Зейдель, бус
хэмжээнд)
Function SpMV(val() As Complex, col() As Integer, ptr() As Integer, U() As Complex) As
Complex()
    Dim n As Integer = ptr.Length - 1
    Dim I(n - 1) As Complex
    For r As Integer = 0 To n - 1
        Dim s As Complex = Complex.Zero
        For p As Integer = ptr(r) To ptr(r + 1) - 1      ' зөвхөн тэг биш элемент
            s += val(p) * U(col(p))
        Next
        I(r) = s
    Next
    Return I
End Function
```

Код 3.2. CSR матриц-векторын үржвэр (VB.NET)

```
def min_degree(adj):
    # Тиннигийн 2-р схем: adj — зангилаа бүрийн хөршийн олонлог (dict of set)
    adj = {v: set(nb) for v, nb in adj.items()}; order = []; fill = 0
    while adj:
        v = min(adj, key=lambda u: (len(adj[u]), u))   # хамгийн бага зэрэг
        nb = adj.pop(v); order.append(v)
        for a in nb:                                    # хөршүүдийг хооронд нь холбох
            adj[a].discard(v)
            for c in nb:
                if c != a and c not in adj[a]:
                    adj[a].add(c); fill += 1
    return order, fill // 2                             # дүүргэлтийн шинэ салааны тоо
```

Код 3.3. Хамгийн бага зэргийн эрэмбэлэлт (Python)

Код 3.3 нь Зураг 3.2-ын блок схемийг шууд хэрэгжүүлсэн: зангилааг хасах бүрт түүний хөршүүдийг хооронд нь холбож (дүүргэлт), зэргийг шинэчилнэ. Хамгийн бага зэрэгтэйг олохдоо энгийн хайлт хийсэн тул $O(n^2)$; практик программд зэргээр нь жагсаасан «саг» (bucket) бүтэц ашиглаж $O(n \log n)$ болгоно.

9. Бие даан гүйцэтгэх программчлалын даалгавар

1. Зураг 3.1-ээр салааны жагсаалтаас CSR бүтэц байгуулж, Y-ийг бүтэн матрицтай харьцуулан шалга.
2. Зураг 3.2-ын Тиннигийн 2-р схемийг бичиж, Жишээ 3.3-ын дарааллыг гарга.
3. Зураг 3.3-ын мөрөөр хийх сийрэг LU-г бичиж, хүчин зүйлийн хүснэгтийг файлд хадгал.
4. Зураг 3.4-ийн сийрэг векторын орлуулгыг ашиглан Z-ийн нэг баганыг зөвхөн шаардлагатай элементээр тооцож, үйлдлийн тоог бүтэн орлуулгатай харьцуул.
5. Хүснэгт 3.5-ын туршилтыг өөрийн эрэмбэлэлтийн программаар давт.

Нэр томьёо

Монгол	Англи	Орос
Сийрэг матриц	Sparse matrix	Разреженная матрица
Сийрэгшил	Sparsity	Разреженность
Шахсан мөрийн схем	Compressed sparse row (CSR)	Сжатое строчное хранение
Холбоост жагсаалт	Linked list	Связный список
Дүүргэлт	Fill-in	Заполнение
Элиминацийн граф	Elimination graph	Граф исключения
Оновчтой эрэмбэлэлт	Optimal ordering	Оптимальное упорядочение
Хамгийн бага зэрэг	Minimum degree	Минимальная степень
Бэлгэдлийн задаргаа	Symbolic factorization	Символьное разложение
Хүчин зүйлийн хүснэгт	Factor table	Таблица множителей
Сийрэг вектор	Sparse vector	Разреженный вектор
Элиминацийн мод	Elimination tree	Дерево исключения

Хүснэгт 3.7. Нэр томьёо

Дүгнэлт

- Сүлжээний Y, B, Якобиан матрицын 99%-иас илүү элемент тэг тул зөвхөн тэгээс ялгаатайг хадгалж, тэдгээрээр үйлдэл хийнэ.
- CSR схем нь санах ой ба хурдны хувьд хамгийн тохиромжтой; холбоост жагсаалт дүүргэлт оруулахад уян хатан.
- Задаргааны дүүргэлт элиминацийн дарааллаас хамаарна: бага зэрэгтэй зангилааг эхэлж хасах нь дүүргэлтийг эрс бууруулна.
- Тиннигийн 2-р схем (хамгийн бага зэрэг) практикт хамгийн өргөн хэрэглэгддэг.
- Эрэмбэлэлт, бэлгэдлийн задаргааг нэг удаа, тоон задаргааг давталт бүрт хийснээр сийрэг LU-ийн үйлдэл n-тэй шугаман хамааралтай болно.

Хяналтын асуулт

1. Сүлжээний матриц яагаад сийрэг байдаг вэ?
2. COO, CSR, холбоост жагсаалтын давуу ба сул талыг харьцуул.
3. CSR-ээр матриц-вектор үржвэрийн алгоритмыг бич.
4. Дүүргэлт гэж юу вэ? Графаар тайлбарла.
5. Тиннигийн гурван схемийн ялгаа?
6. Бэлгэдлийн ба тоон задаргааг яагаад салгадаг вэ?
7. Хүчин зүйлийн хүснэгт гэж юу вэ?
8. Сийрэг векторын арга ямар бодлогод ашигтай вэ?
9. Сийрэг LU-ийн үйлдлийн тоо яагаад n-тэй шугаман хамааралтай вэ?
10. Од хэлбэрийн сүлжээнд төв зангилааг эхэлж хасвал юу болох вэ?
11. Y матрицын тэгш хэмт шинжийг хадгалахад хэрхэн ашиглах вэ?
12. Сийрэг LU-д гол элемент сонголт яагаад асуудал үүсгэдэг вэ?

Бие даалтын бодлого

1. n = 200, m = 300 салаатай сүлжээний Y-ийн nnz ба сийрэгшил? *Хариу: 800; 98%.*
2. Бодлого 1-ийн CSR-ийн санах ой (val 8, col 4, rp 4 байт)? *Хариу: 800·12 + 201·4 = 10 404 байт.*
3. Дүүрэн хадгалалтын санах ой (Бодлого 1, 8 байт)? *Хариу: 320 000 байт.*
4. Зам хэлбэрийн сүлжээ 1–2–3–4–5-ийг 3, 1, 2, 4, 5 дарааллаар задлахад дүүргэлт? *Хариу: 1 ((2, 4)).*
5. Бодлого 4-ийн хамгийн бага зэргийн дараалал ба дүүргэлт? *Хариу: 1, 2, 3, 4, 5 эсвэл 5, 4, …; 0.*
6. Цагираг 1–2–3–4–1-ийг 1, 2, 3, 4 дарааллаар задлахад дүүргэлт? *Хариу: 1 ((2, 4)).*
7. Од (төв 1, хөрш 2…6)-ийг төвөөс эхлэхэд дүүргэлт? *Хариу: C(5,2) = 10.*
8. Бодлого 7-д төвийг сүүлд хасвал? *Хариу: 0.*

9. Жишээ 3.1-ийн B-ийн 2-р мөрийг CSR-ээс уншихад val-ийн аль индексүүд? *Хариу: 6, 7, 8.*
10. Жишээ 3.3-т анхны дарааллын L + U-ийн nnz (диагональтай)? *Хариу: 32.*
11. Баганын элементийн тоо $c_k = 4$ бүх k-д, n = 5000 бол сийрэг LU-ийн үйлдлийн тоо? *Хариу: ≈ $8 \cdot 10^4$.*
12. Бодлого 11-д дүүрэн LU-тэй харьцуулбал хэд дахин хурдан? *Хариу: ≈ $5 \cdot 10^5$ дахин.*
13. x = (1; 2; 0; 0; 1)-ийг Жишээ 3.1-ийн B-ээр CSR алгоритмаар үржүүл. *Хариу: (−1; 26; −21; −4; 0).*
14. Жишээ 3.1-д 4, 5, 1, 2, 3 дарааллаар задлахад L-ийн тэгээс ялгаатай (диагональгүй) элемент? *Хариу: 5.*
15. IEEE 118 (n = 118, m = 186)-ийн дундаж зэрэг? *Хариу: $2m/n \approx 3.15$.*

Ашигласан ном

1. W. F. Tinney, J. W. Walker. Direct solutions of sparse network equations by optimally ordered triangular factorization // Proc. IEEE, vol. 55, no. 11, 1967.
2. T. A. Davis. Direct Methods for Sparse Linear Systems. — SIAM, 2006.
3. Y. Saad. Iterative Methods for Sparse Linear Systems. 2nd ed. — SIAM, 2003.
4. G. H. Golub, C. F. Van Loan. Matrix Computations. 4th ed. — Johns Hopkins University Press, 2013.
5. B. Stott, O. Alsaç. Fast decoupled load flow // IEEE Trans. PAS, vol. 93, 1974.
6. R. D. Zimmerman, C. E. Murillo-Sánchez, R. J. Thomas. MATPOWER: steady-state operations, planning and analysis tools for power systems research and education // IEEE Trans. Power Systems, vol. 26, no. 1, pp. 12–19, 2011.
7. Ш.Гантөмөр. «Горим 5.0» (PowerPlan) — цахилгаан сүлжээний тогтсон горим, алдагдлын тооцооны программ: аргачлалын тайлбар. — ШУТИС, 2026.